

## Summary

This briefing document synthesizes the core principles of asymptotic analysis as presented in the instructional session by Pankaj Sharma. The primary focus is the transition from exact analysis to approximate analysis, specifically evaluating algorithm running time in relation to input size ( $n$ ). The document outlines the identification of primitive operations, the mathematical foundations of Big O notation, and the relative growth rates of various functional classes (logarithmic, polynomial, exponential, and factorial). Key takeaways include the necessity of focusing on large-scale inputs ( $n \rightarrow \infty$ ), the irrelevance of constant factors and lower-order terms in growth analysis, and the formal methodology for proving asymptotic bounds.

### 1. Foundations of Asymptotic Analysis

Asymptotic analysis serves as a framework for categorizing functions based on their growth rates rather than their exact values for specific inputs. The source emphasizes two critical factors in this analysis:

- **Running Time in Terms of Input Size:** The efficiency of an algorithm is determined by how its execution time increases as the input size ( $n$ ) grows.
- **Primitive Operations:** Analysis is conducted by identifying and counting fundamental operations (multiplication, addition, division, assignments, comparisons, increments, and jumps) rather than measuring real-time execution in seconds, which can vary by hardware.

#### Contextual Note on Educational Consistency

The source highlights the importance of consistency in learning, noting that students often struggle with algorithms due to a lack of mathematical rigor or inconsistent attendance. Additionally, it mentions a contemporary Unacademy promotional offer (50% off and "double validity" on 18-month courses) as part of a recurring educational support cycle.

### 2. Comparative Analysis: Constant vs. Linear Time

A central theme is the comparison between two algorithmic approaches to the same problem: calculating the sum of squares up to  $n$ .

#### Algorithm 1: The Mathematical Formula

Utilizing the formula ( $\frac{n(n+1)(2n+1)}{6}$ ), this approach involves a fixed number of operations:

- **Operations:** 3 Multiplications, 2 Additions, 1 Division, 1 Jump.
- **Analysis:** Regardless of whether  $n$  is 10 or 500, the number of primitive operations remains the same.

- Conclusion: This algorithm exhibits Constant Growth; its running time does not change relative to input size.

#### Algorithm 2: The Iterative Loop

Using a standard for loop to accumulate the sum involves operations that repeat based on  $n$ .

- Operations: 1 Initial Assignment,  $n+1$  Comparisons,  $n$  Increments,  $n$  Additions,  $n$  Multiplications, and  $n$  Assignments inside the loop.
- Analysis: The total time is expressed as  $(C \cdot n + C')$ .
- Conclusion: This algorithm exhibits Linear Growth. As  $n$  increases, the running time increases proportionally.

| Algorithm Type | Complexity Class | Growth Behavior           |
|----------------|------------------|---------------------------|
| Formula-based  | Constant         | No growth relative to $n$ |
| Loop-based     | Linear           | Growth increases with $n$ |

### 3. Big O Notation: Principles and Mathematical Basis

Big O notation ( $O$ ) is used to provide an upper bound on the growth rate of a function. The statement  $(f(n) = O(g(n)))$  implies that the growth of  $(g(n))$  is either the same as or greater than the growth of  $(f(n))$ .

#### Key Analytical Rules

1. Drop Lower-Order Terms: In a function like  $(3n^2 + 10n + 2)$ , the terms  $10n$  and  $2$  are ignored as  $n$  becomes very large.
2. Ignore Constants: The coefficient (e.g.,  $3$  in  $(3n)$ ) does not change the growth category.  $(2n)$  and  $(200n)$  are both considered to have the same linear growth rate.
3. Focus on Large Input ( $n \rightarrow \infty$ ): Algorithms are not designed for small inputs. While a small input might suggest one algorithm is faster, asymptotic analysis only considers behavior as  $n$  approaches infinity.

#### Formal Proof Methodology

To prove  $(f(n) = O(g(n)))$ , one must find a positive constant  $C$  and a starting point  $(n_0)$  such that:

$$[f(n) \leq C \cdot g(n) \quad \text{for all} \quad n \geq n_0.]$$

- Example: To prove  $(2^n = O(3^n))$ , we find that  $[\frac{2^n}{3^n} \leq C, ]$  which simplifies to  $(\left(\frac{2}{3}\right)^n \leq C)$ . Since  $(\frac{2}{3} < 1)$ , as  $n$  increases, the value decreases, making it easy to find a constant  $C$  (e.g.,  $C=1$ ).

- Counter-Example: One cannot prove  $(3^n = O(2^n))$  because the ratio  $(\left(\frac{3}{2}\right)^n)$  grows infinitely, meaning no fixed constant  $C$  can bound it for all  $n$ .

#### 4. The Hierarchy of Functional Growth

The source provides a comprehensive ranking of functions from slowest growth (most efficient) to fastest growth (least efficient).

##### Relative Growth Categories

1. Logarithmic Functions:  $(\log(\log n) < \log n < (\log n)^2)$ . These grow very slowly and are generally more efficient than polynomial functions.
2. Polynomial Functions:  $(\sqrt[n]{n}) (n^{0.5}) < n < (n^2) < (n^3)$ . Within this group, a higher degree signifies faster growth.
3. Exponential Functions:  $(2^n < 3^n < 4^n)$ . These grow much faster than any polynomial function.
4. Factorial and Beyond:  $(n! < n^n)$ . Factorial growth is extremely rapid, and  $(n^n)$  represents one of the highest common growth rates in algorithmic analysis.

##### Summary Hierarchy

The following sequence represents the non-decreasing order of growth for standard functions:  
 $[\log(\log n) \rightarrow \log n \rightarrow (\log n)^k \rightarrow n^k, (\text{Polynomial}) \rightarrow a^n, (\text{Exponential}) \rightarrow n! \rightarrow n^n]$

#### 5. Critical Observations on Mathematical Intuition

- Logarithmic vs. Polynomial: Even if a logarithmic function has a high power (e.g.,  $(\log n)^{10})$ , it will eventually be overtaken by a polynomial function (e.g.,  $n$  or  $(\sqrt[n]{n})$ ) as  $n \rightarrow \infty$ .
- Exponential Comparisons:  $(2^n)$  is significantly smaller than  $(2^{2n})$ . This is because  $(2^{2n})$  is mathematically equivalent to  $((2^n)^2)$ . If  $(2^n)$  is treated as  $x$ , the comparison is between  $x$  and  $(x^2)$ .
- The Factorial logic:  $(n!)$  (the product of 1 to  $n$ ) is bounded above by  $(n^n)$  (the product of  $n$  repeated  $n$  times), confirming that  $(n! = O(n^n))$ .

#### 6. Closing Insights

The analysis concludes that understanding algorithm complexity requires moving past "rote memorization" of notations and instead developing a deep intuition for mathematical growth. By focusing on the highest-order term and treating  $n$  as a value approaching infinity, one can effectively categorize any algorithm's efficiency within the established asymptotic hierarchy.

